

air_quality_autoencoder

April 30, 2025

1 Reducción de Dimensionalidad de Datos de Calidad del Aire Usando Autoencoders

Este notebook utiliza autoencoders para reducir la dimensionalidad de los datos de calidad del aire de la red RAMA en la Ciudad de México. Analizamos datos de tres estaciones (BJU, UAX, MER) y tres contaminantes (CO, O3, NO2) durante el período 2020-2024.

Autor: Thanos Drossos

1.1 Resumen

1. Importar y preprocesar datos de archivos XLS (similar a AirQuality.ipynb)
2. Crear un conjunto de datos con 9 características (3 contaminantes $\times$ 3 estaciones)
3. Implementar un autoencoder usando TensorFlow/Keras para reducir dimensiones a 2D
4. Visualizar los resultados en un gráfico 2D

Este enfoque nos permite identificar patrones y relaciones en los datos que podrían no ser evidentes en el espacio original de 9 dimensiones.

2 Resultados y Conclusiones

En este notebook, utilizamos un autoencoder para reducir la dimensionalidad de los datos de calidad del aire de 9 características (3 contaminantes $\times$ 3 estaciones) a 2 dimensiones. La visualización de los datos codificados revela patrones relacionados con el tiempo (año y mes) que podrían ser difíciles de identificar en el espacio original de alta dimensionalidad.

2.1 Hallazgos principales después de la reducción de dimensiones:

1. Patrones temporales evidentes: Se observan claros patrones temporales al comparar por año y por mes, sugiriendo ciclos estacionales en la contaminación del aire.
2. Homogeneidad entre estaciones: No se aprecian diferencias significativas entre las estaciones en el espacio 2D, a pesar de que los datos originales difieren en magnitud. Esto sugiere que los patrones de contaminación son similares entre estaciones, aunque con diferentes intensidades.
3. Error de reconstrucción por contaminante: El CO de las tres estaciones presenta el mayor error de reconstrucción, lo que significa que este contaminante es el menos bien representado en el espacio 2D y posiblemente tiene comportamientos más complejos o independientes que no pueden capturarse completamente en solo dos dimensiones.

4. Diferencias geográficas: Las estaciones de UAX (Xochimilco), ubicadas en la periferia, muestran un error de reconstrucción aumentado en comparación con las otras dos estaciones más céntricas (Benito Juárez y Merced). Esto sugiere que la dinámica de contaminación en zonas periféricas podría ser más compleja.
5. Correlaciones diferenciales: Los datos de O₃ son muy diferentes a los de NO_x y CO en términos de correlación con las dimensiones codificadas. Esto refleja la naturaleza distinta del ozono como contaminante secundario, formado por reacciones fotoquímicas.
6. Agrupación por tipo de contaminante: Al visualizar por contaminante, se observa claramente la similitud entre NO_x y CO (contaminantes primarios relacionados con combustión), mientras que el O₃ aparece aislado, confirmando su comportamiento diferenciado.

```
[ ]: import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from glob import glob
from scipy.stats import pearsonr, spearmanr
from sklearn.feature_selection import mutual_info_regression
from sklearn.preprocessing import StandardScaler

# Para autoencoders
import tensorflow as tf
from tensorflow import keras

model = keras.models.Model
sequential = keras.models.Sequential
dense = keras.layers.Dense
input_layer = keras.layers.Input
activation = keras.layers.Activation

np.random.seed(69)
tf.random.set_seed(69)
```

2.2 1. Carga y Preprocesamiento de Datos

Primero, cargaremos y preprocesaremos los datos de calidad del aire de los archivos XLS siguiendo el mismo enfoque que en AirQuality.ipynb.

```
[ ]: contaminants = ['CO', 'O3', 'NO2']
years = range(2020, 2025)
stations = ['BJU', 'UAX', 'MER']

data_dir = 'RAMA'

data_by_contaminant = {contaminant: {} for contaminant in contaminants}
```

```

for contaminant in contaminants:
    dfs = []
    for year in years:
        filename = os.path.join(data_dir, f"{year}-{contaminant}.xls")
        if os.path.exists(filename):
            print(f"Procesando {filename}")
            try:
                df = pd.read_excel(filename)

                # Filtrar solo las estaciones requeridas
                station_columns = [col for col in df.columns if col in stations]
                if not station_columns:
                    print(f"No se encontraron estaciones requeridas en {filename}")
                    continue

                columns_to_keep = ['FECHA', 'HORA'] + station_columns
                df = df[columns_to_keep]

                df = df.replace(-99, np.nan)

                df['YEAR'] = year
                dfs.append(df)
            except Exception as e:
                print(f"Error al procesar {filename}: {e}")

    if dfs:
        # Combinar todos los datos para este contaminante
        combined_df = pd.concat(dfs, ignore_index=True)

        combined_df['DATETIME'] = pd.to_datetime(combined_df['FECHA'],
        ↪format='%d.%m.%Y', errors='coerce')
        combined_df['DATE'] = combined_df['DATETIME'].dt.date

        data_by_contaminant[contaminant] = combined_df
    else:
        print(f"No se encontraron datos para {contaminant}")

```

```

Procesando RAMA\2020C0.xls
Procesando RAMA\2021C0.xls
Procesando RAMA\2022C0.xls
Procesando RAMA\2023C0.xls
Procesando RAMA\2024C0.xls
Procesando RAMA\202003.xls
Procesando RAMA\202103.xls
Procesando RAMA\202203.xls
Procesando RAMA\202303.xls

```

```

Procesando RAMA\202403.xls
Procesando RAMA\2020N02.xls
Procesando RAMA\2021N02.xls
Procesando RAMA\2022N02.xls
Procesando RAMA\2023N02.xls
Procesando RAMA\2024N02.xls

```

2.3 2. Calcular Promedios Diarios y Crear Conjunto de Datos Combinado

A continuación, calcularemos promedios diarios para cada estación y contaminante, y luego los combinaremos en un único conjunto de datos para el autoencoder.

```

[3]: daily_avg_by_contaminant = {}

for contaminant, df in data_by_contaminant.items():
    if not df.empty:
        daily_avg = df.groupby('DATE')[stations].mean()
        daily_avg_by_contaminant[contaminant] = daily_avg

        print(f"Promedios diarios para {contaminant}:")
        print(daily_avg.head())

combined_daily = pd.DataFrame()

for contaminant, daily_avg in daily_avg_by_contaminant.items():
    for station in stations:
        if station in daily_avg.columns:
            col_name = f"{contaminant}_{station}"
            combined_daily[col_name] = daily_avg[station]

print("\nConjunto de datos combinado:")
print(combined_daily.head())
print(f"\nDimensiones del conjunto de datos: {combined_daily.shape}")

```

Promedios diarios para CO:

	BJU	UAX	MER
DATE			
2020-01-01	0.458333	NaN	0.630435
2020-01-02	0.128571	NaN	0.242857
2020-01-03	0.270833	NaN	0.491667
2020-01-04	0.304167	0.295833	0.454167
2020-01-05	0.520833	0.445833	0.391667

Promedios diarios para O3:

	BJU	UAX	MER
DATE			
2020-01-01	16.521739	23.652174	18.086957
2020-01-02	16.047619	21.666667	15.285714
2020-01-03	21.375000	27.454545	19.250000

2020-01-04	23.764706	28.750000	21.666667
2020-01-05	NaN	28.541667	24.916667

Promedios diarios para NO2:

	BJU	UAX	MER
DATE			
2020-01-01	16.086957	9.000000	16.521739
2020-01-02	9.190476	3.904762	12.761905
2020-01-03	22.583333	12.100000	29.541667
2020-01-04	28.833333	NaN	37.791667
2020-01-05	36.166667	NaN	42.166667

Conjunto de datos combinado:

	CO_BJU	CO_UAX	CO_MER	O3_BJU	O3_UAX	O3_MER	\
DATE							
2020-01-01	0.458333	NaN	0.630435	16.521739	23.652174	18.086957	
2020-01-02	0.128571	NaN	0.242857	16.047619	21.666667	15.285714	
2020-01-03	0.270833	NaN	0.491667	21.375000	27.454545	19.250000	
2020-01-04	0.304167	0.295833	0.454167	23.764706	28.750000	21.666667	
2020-01-05	0.520833	0.445833	0.391667	NaN	28.541667	24.916667	

	NO2_BJU	NO2_UAX	NO2_MER
DATE			
2020-01-01	16.086957	9.000000	16.521739
2020-01-02	9.190476	3.904762	12.761905
2020-01-03	22.583333	12.100000	29.541667
2020-01-04	28.833333	NaN	37.791667
2020-01-05	36.166667	NaN	42.166667

Dimensiones del conjunto de datos: (1674, 9)

2.4 3. Preparar Datos para el Autoencoder

Necesitamos preparar los datos para el autoencoder mediante: 1. Manejo de valores faltantes 2. Estandarización de las características 3. Creación de conjuntos de entrenamiento y prueba

```
[4]: combined_daily_cleaned = combined_daily.dropna()
print(f"Dimensiones después de eliminar filas con valores faltantes:\n
      ↳{combined_daily_cleaned.shape}")

# Estandarizar los datos (media=0, desv=1)
scaler = StandardScaler()
scaled_data = scaler.fit_transform(combined_daily_cleaned)

# Dividir datos en conjuntos de entrenamiento y prueba (80% entrenamiento, 20%
↳prueba)
train_size = int(0.8 * len(scaled_data))
train_data = scaled_data[:train_size]
test_data = scaled_data[train_size:]
```

```
print(f"Dimensiones de datos de entrenamiento: {train_data.shape}")
print(f"Dimensiones de datos de prueba: {test_data.shape}")
```

Dimensiones después de eliminar filas con valores faltantes: (1197, 9)

Dimensiones de datos de entrenamiento: (957, 9)

Dimensiones de datos de prueba: (240, 9)

2.5 4. Construir y Entrenar el Autoencoder

Ahora construiremos una red neuronal de autoencoder para reducir la dimensionalidad de nuestros datos de 9 características a 2 dimensiones.

```
[5]: input_dim = train_data.shape[1] # Debería ser 9 (3 contaminantes × 3
    ↪ estaciones)

input_data = input_layer(shape=(input_dim,))

encoder = dense(6)(input_data)
encoder = activation('relu')(encoder)

encoder = dense(4)(encoder)
encoder = activation('relu')(encoder)

encoded = dense(2, name='bottleneck')(encoder)

decoder = dense(4)(encoded)
decoder = activation('relu')(decoder)

decoder = dense(6)(decoder)
decoder = activation('relu')(decoder)

decoded = dense(input_dim)(decoder)

autoencoder = model(inputs=input_data, outputs=decoded)
autoencoder.compile(loss='mse', optimizer='adam')

autoencoder.summary()
```

Model: "model"

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 9)]	0
dense (Dense)	(None, 6)	60
activation (Activation)	(None, 6)	0

dense_1 (Dense)	(None, 4)	28
activation_1 (Activation)	(None, 4)	0
bottleneck (Dense)	(None, 2)	10
dense_2 (Dense)	(None, 4)	12
activation_2 (Activation)	(None, 4)	0

Layer (type)	Output Shape	Param #
=====		
input_1 (InputLayer)	[(None, 9)]	0
dense (Dense)	(None, 6)	60
activation (Activation)	(None, 6)	0
dense_1 (Dense)	(None, 4)	28
activation_1 (Activation)	(None, 4)	0
bottleneck (Dense)	(None, 2)	10
dense_2 (Dense)	(None, 4)	12
activation_2 (Activation)	(None, 4)	0
dense_3 (Dense)	(None, 6)	30
activation_3 (Activation)	(None, 6)	0
dense_4 (Dense)	(None, 9)	63
=====		
Total params: 203		
Trainable params: 203		
Non-trainable params: 0		

```
[6]: history = autoencoder.fit(
    train_data, train_data, # Entrada = Salida
    epochs=100,
    batch_size=32,
    shuffle=True,
    validation_data=(test_data, test_data),
```

```
    verbose=1
)
```

```
Epoch 1/100
30/30 [=====] - 1s 8ms/step - loss: 0.9295 - val_loss:
1.2876
Epoch 2/100
30/30 [=====] - 0s 2ms/step - loss: 0.9128 - val_loss:
1.2828
Epoch 3/100
30/30 [=====] - 0s 2ms/step - loss: 0.8932 - val_loss:
1.2735
Epoch 4/100
30/30 [=====] - 0s 2ms/step - loss: 0.8616 - val_loss:
1.2611
Epoch 5/100
30/30 [=====] - 0s 2ms/step - loss: 0.8305 - val_loss:
1.2465
Epoch 6/100
30/30 [=====] - 0s 2ms/step - loss: 0.7977 - val_loss:
1.2217
Epoch 7/100
30/30 [=====] - 0s 2ms/step - loss: 0.7515 - val_loss:
1.1811
Epoch 8/100
30/30 [=====] - 0s 2ms/step - loss: 0.6908 - val_loss:
1.1290
Epoch 9/100
30/30 [=====] - 0s 2ms/step - loss: 0.6322 - val_loss:
1.0765
Epoch 10/100
30/30 [=====] - 0s 3ms/step - loss: 0.5705 - val_loss:
1.0058
Epoch 11/100
30/30 [=====] - 0s 2ms/step - loss: 0.4960 - val_loss:
0.9282
Epoch 12/100
30/30 [=====] - 0s 2ms/step - loss: 0.4303 - val_loss:
0.8538
Epoch 13/100
30/30 [=====] - 0s 2ms/step - loss: 0.3765 - val_loss:
0.7869
Epoch 14/100
30/30 [=====] - 0s 2ms/step - loss: 0.3348 - val_loss:
0.7268
Epoch 15/100
30/30 [=====] - 0s 2ms/step - loss: 0.3018 - val_loss:
0.6682
```


Epoch 16/100
30/30 [=====] - 0s 2ms/step - loss: 0.2767 - val_loss: 0.6139
Epoch 17/100
30/30 [=====] - 0s 2ms/step - loss: 0.2558 - val_loss: 0.5672
Epoch 18/100
30/30 [=====] - 0s 2ms/step - loss: 0.2412 - val_loss: 0.5227
Epoch 19/100
30/30 [=====] - 0s 2ms/step - loss: 0.2267 - val_loss: 0.4897
Epoch 20/100
30/30 [=====] - 0s 2ms/step - loss: 0.2163 - val_loss: 0.4549
Epoch 21/100
30/30 [=====] - 0s 3ms/step - loss: 0.2066 - val_loss: 0.4282
Epoch 22/100
30/30 [=====] - 0s 2ms/step - loss: 0.1976 - val_loss: 0.3995
Epoch 23/100
30/30 [=====] - 0s 2ms/step - loss: 0.1901 - val_loss: 0.3776
Epoch 24/100
30/30 [=====] - 0s 2ms/step - loss: 0.1841 - val_loss: 0.3577
Epoch 25/100
30/30 [=====] - 0s 2ms/step - loss: 0.1793 - val_loss: 0.3424
Epoch 26/100
30/30 [=====] - 0s 2ms/step - loss: 0.1755 - val_loss: 0.3301
Epoch 27/100
30/30 [=====] - 0s 2ms/step - loss: 0.1721 - val_loss: 0.3197
Epoch 28/100
30/30 [=====] - 0s 2ms/step - loss: 0.1691 - val_loss: 0.3080
Epoch 29/100
30/30 [=====] - 0s 2ms/step - loss: 0.1667 - val_loss: 0.2971
Epoch 30/100
30/30 [=====] - 0s 2ms/step - loss: 0.1645 - val_loss: 0.2893
Epoch 31/100
30/30 [=====] - 0s 2ms/step - loss: 0.1625 - val_loss: 0.2849

Epoch 32/100
30/30 [=====] - 0s 2ms/step - loss: 0.1611 - val_loss: 0.2772

Epoch 33/100
30/30 [=====] - 0s 2ms/step - loss: 0.1595 - val_loss: 0.2718

Epoch 34/100
30/30 [=====] - 0s 2ms/step - loss: 0.1585 - val_loss: 0.2670

Epoch 35/100
30/30 [=====] - 0s 2ms/step - loss: 0.1574 - val_loss: 0.2624

Epoch 36/100
30/30 [=====] - 0s 2ms/step - loss: 0.1562 - val_loss: 0.2592

Epoch 37/100
30/30 [=====] - 0s 2ms/step - loss: 0.1554 - val_loss: 0.2546

Epoch 38/100
30/30 [=====] - 0s 2ms/step - loss: 0.1543 - val_loss: 0.2522

Epoch 39/100
30/30 [=====] - 0s 3ms/step - loss: 0.1537 - val_loss: 0.2496

Epoch 40/100
30/30 [=====] - 0s 2ms/step - loss: 0.1530 - val_loss: 0.2481

Epoch 41/100
30/30 [=====] - 0s 2ms/step - loss: 0.1523 - val_loss: 0.2435

Epoch 42/100
30/30 [=====] - 0s 2ms/step - loss: 0.1515 - val_loss: 0.2423

Epoch 43/100
30/30 [=====] - 0s 2ms/step - loss: 0.1509 - val_loss: 0.2392

Epoch 44/100
30/30 [=====] - 0s 2ms/step - loss: 0.1502 - val_loss: 0.2378

Epoch 45/100
30/30 [=====] - 0s 2ms/step - loss: 0.1497 - val_loss: 0.2358

Epoch 46/100
30/30 [=====] - 0s 2ms/step - loss: 0.1492 - val_loss: 0.2334

Epoch 47/100
30/30 [=====] - 0s 2ms/step - loss: 0.1487 - val_loss: 0.2316

Epoch 48/100
30/30 [=====] - 0s 2ms/step - loss: 0.1481 - val_loss: 0.2312
Epoch 49/100
30/30 [=====] - 0s 2ms/step - loss: 0.1476 - val_loss: 0.2290
Epoch 50/100
30/30 [=====] - 0s 2ms/step - loss: 0.1471 - val_loss: 0.2278
Epoch 51/100
30/30 [=====] - 0s 2ms/step - loss: 0.1466 - val_loss: 0.2265
Epoch 52/100
30/30 [=====] - 0s 2ms/step - loss: 0.1461 - val_loss: 0.2250
Epoch 53/100
30/30 [=====] - 0s 2ms/step - loss: 0.1458 - val_loss: 0.2244
Epoch 54/100
30/30 [=====] - 0s 2ms/step - loss: 0.1453 - val_loss: 0.2229
Epoch 55/100
30/30 [=====] - 0s 2ms/step - loss: 0.1451 - val_loss: 0.2218
Epoch 56/100
30/30 [=====] - 0s 2ms/step - loss: 0.1446 - val_loss: 0.2207
Epoch 57/100
30/30 [=====] - 0s 2ms/step - loss: 0.1444 - val_loss: 0.2196
Epoch 58/100
30/30 [=====] - 0s 2ms/step - loss: 0.1441 - val_loss: 0.2185
Epoch 59/100
30/30 [=====] - 0s 2ms/step - loss: 0.1437 - val_loss: 0.2174
Epoch 60/100
30/30 [=====] - 0s 2ms/step - loss: 0.1435 - val_loss: 0.2167
Epoch 61/100
30/30 [=====] - 0s 2ms/step - loss: 0.1432 - val_loss: 0.2166
Epoch 62/100
30/30 [=====] - 0s 2ms/step - loss: 0.1430 - val_loss: 0.2156
Epoch 63/100
30/30 [=====] - 0s 2ms/step - loss: 0.1428 - val_loss: 0.2148

Epoch 64/100
30/30 [=====] - 0s 2ms/step - loss: 0.1427 - val_loss: 0.2148
Epoch 65/100
30/30 [=====] - 0s 2ms/step - loss: 0.1426 - val_loss: 0.2139
Epoch 66/100
30/30 [=====] - 0s 2ms/step - loss: 0.1423 - val_loss: 0.2124
Epoch 67/100
30/30 [=====] - 0s 2ms/step - loss: 0.1422 - val_loss: 0.2122
Epoch 68/100
30/30 [=====] - 0s 2ms/step - loss: 0.1420 - val_loss: 0.2113
Epoch 69/100
30/30 [=====] - 0s 2ms/step - loss: 0.1418 - val_loss: 0.2109
Epoch 70/100
30/30 [=====] - 0s 3ms/step - loss: 0.1416 - val_loss: 0.2112
Epoch 71/100
30/30 [=====] - 0s 2ms/step - loss: 0.1415 - val_loss: 0.2102
Epoch 72/100
30/30 [=====] - 0s 2ms/step - loss: 0.1414 - val_loss: 0.2108
Epoch 73/100
30/30 [=====] - 0s 2ms/step - loss: 0.1413 - val_loss: 0.2094
Epoch 74/100
30/30 [=====] - 0s 2ms/step - loss: 0.1411 - val_loss: 0.2094
Epoch 75/100
30/30 [=====] - 0s 2ms/step - loss: 0.1409 - val_loss: 0.2092
Epoch 76/100
30/30 [=====] - 0s 2ms/step - loss: 0.1409 - val_loss: 0.2082
Epoch 77/100
30/30 [=====] - 0s 2ms/step - loss: 0.1407 - val_loss: 0.2087
Epoch 78/100
30/30 [=====] - 0s 2ms/step - loss: 0.1405 - val_loss: 0.2081
Epoch 79/100
30/30 [=====] - 0s 2ms/step - loss: 0.1405 - val_loss: 0.2081

Epoch 80/100
30/30 [=====] - 0s 2ms/step - loss: 0.1404 - val_loss: 0.2067

Epoch 81/100
30/30 [=====] - 0s 2ms/step - loss: 0.1404 - val_loss: 0.2076

Epoch 82/100
30/30 [=====] - 0s 2ms/step - loss: 0.1402 - val_loss: 0.2075

Epoch 83/100
30/30 [=====] - 0s 2ms/step - loss: 0.1401 - val_loss: 0.2060

Epoch 84/100
30/30 [=====] - 0s 2ms/step - loss: 0.1400 - val_loss: 0.2062

Epoch 85/100
30/30 [=====] - 0s 2ms/step - loss: 0.1400 - val_loss: 0.2063

Epoch 86/100
30/30 [=====] - 0s 2ms/step - loss: 0.1399 - val_loss: 0.2059

Epoch 87/100
30/30 [=====] - 0s 2ms/step - loss: 0.1399 - val_loss: 0.2060

Epoch 88/100
30/30 [=====] - 0s 2ms/step - loss: 0.1398 - val_loss: 0.2056

Epoch 89/100
30/30 [=====] - 0s 2ms/step - loss: 0.1398 - val_loss: 0.2057

Epoch 90/100
30/30 [=====] - 0s 2ms/step - loss: 0.1398 - val_loss: 0.2050

Epoch 91/100
30/30 [=====] - 0s 2ms/step - loss: 0.1397 - val_loss: 0.2051

Epoch 92/100
30/30 [=====] - 0s 2ms/step - loss: 0.1396 - val_loss: 0.2053

Epoch 93/100
30/30 [=====] - 0s 2ms/step - loss: 0.1396 - val_loss: 0.2047

Epoch 94/100
30/30 [=====] - 0s 2ms/step - loss: 0.1396 - val_loss: 0.2048

Epoch 95/100
30/30 [=====] - 0s 2ms/step - loss: 0.1395 - val_loss: 0.2048

```

Epoch 96/100
30/30 [=====] - 0s 2ms/step - loss: 0.1394 - val_loss: 0.2045
Epoch 97/100
30/30 [=====] - 0s 3ms/step - loss: 0.1394 - val_loss: 0.2045
Epoch 98/100
30/30 [=====] - 0s 2ms/step - loss: 0.1393 - val_loss: 0.2041
Epoch 99/100
30/30 [=====] - 0s 2ms/step - loss: 0.1393 - val_loss: 0.2043
Epoch 100/100
30/30 [=====] - 0s 2ms/step - loss: 0.1394 - val_loss: 0.2036

```

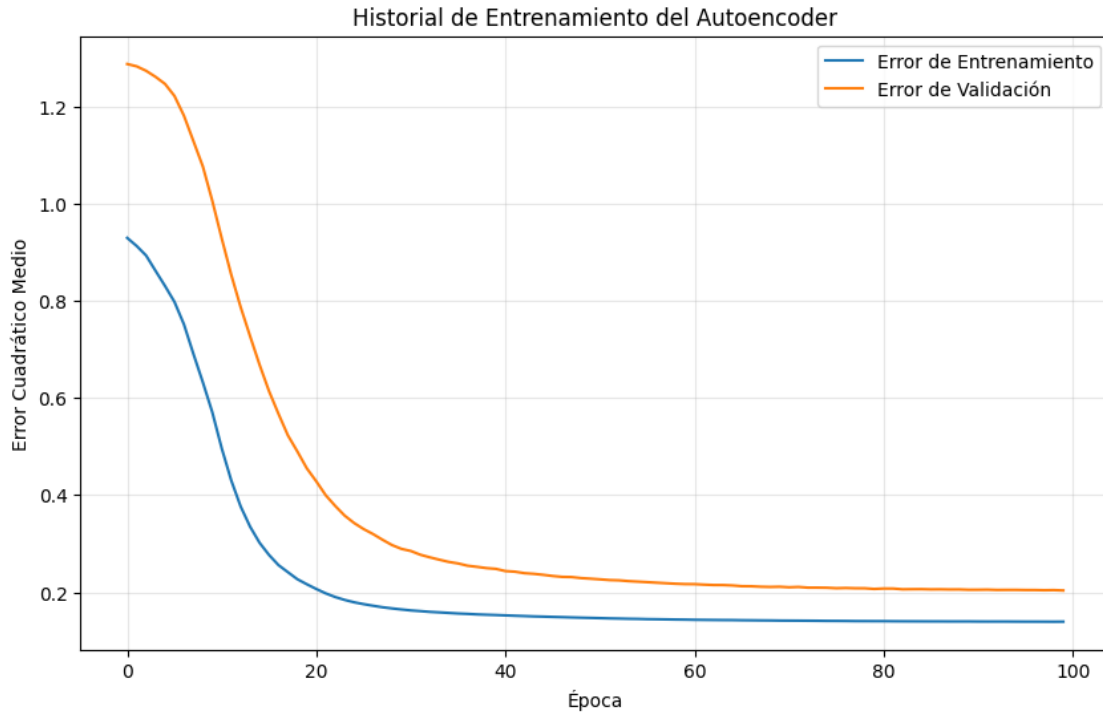
2.6 5. Visualizar Historial de Entrenamiento

Visualicemos el error de entrenamiento y validación para ver qué tan bien se entrenó nuestro modelo.

```

[7]: plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'], label='Error de Entrenamiento')
plt.plot(history.history['val_loss'], label='Error de Validación')
plt.title('Historial de Entrenamiento del Autoencoder')
plt.xlabel('Época')
plt.ylabel('Error Cuadrático Medio')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

```



2.7 6. Extraer y Visualizar la Representación Codificada (2D)

Ahora crearemos un modelo codificador separado para extraer la representación 2D de nuestros datos y visualizarla.

```
[8]: # Crear un modelo codificador para obtener la representación 2D
encoder_model = model(inputs=autoencoder.input, outputs=autoencoder.
    ↪get_layer('bottleneck').output)

encoded_data = encoder_model.predict(scaled_data)

encoded_df = pd.DataFrame()
encoded_df['x'] = encoded_data[:, 0]
encoded_df['y'] = encoded_data[:, 1]

# Añadir información de fecha del conjunto de datos original
encoded_df['date'] = combined_daily_cleaned.index
encoded_df['year'] = pd.to_datetime(encoded_df['date']).dt.year
encoded_df['month'] = pd.to_datetime(encoded_df['date']).dt.month

# Mostrar las primeras filas
print(encoded_df.head())
```

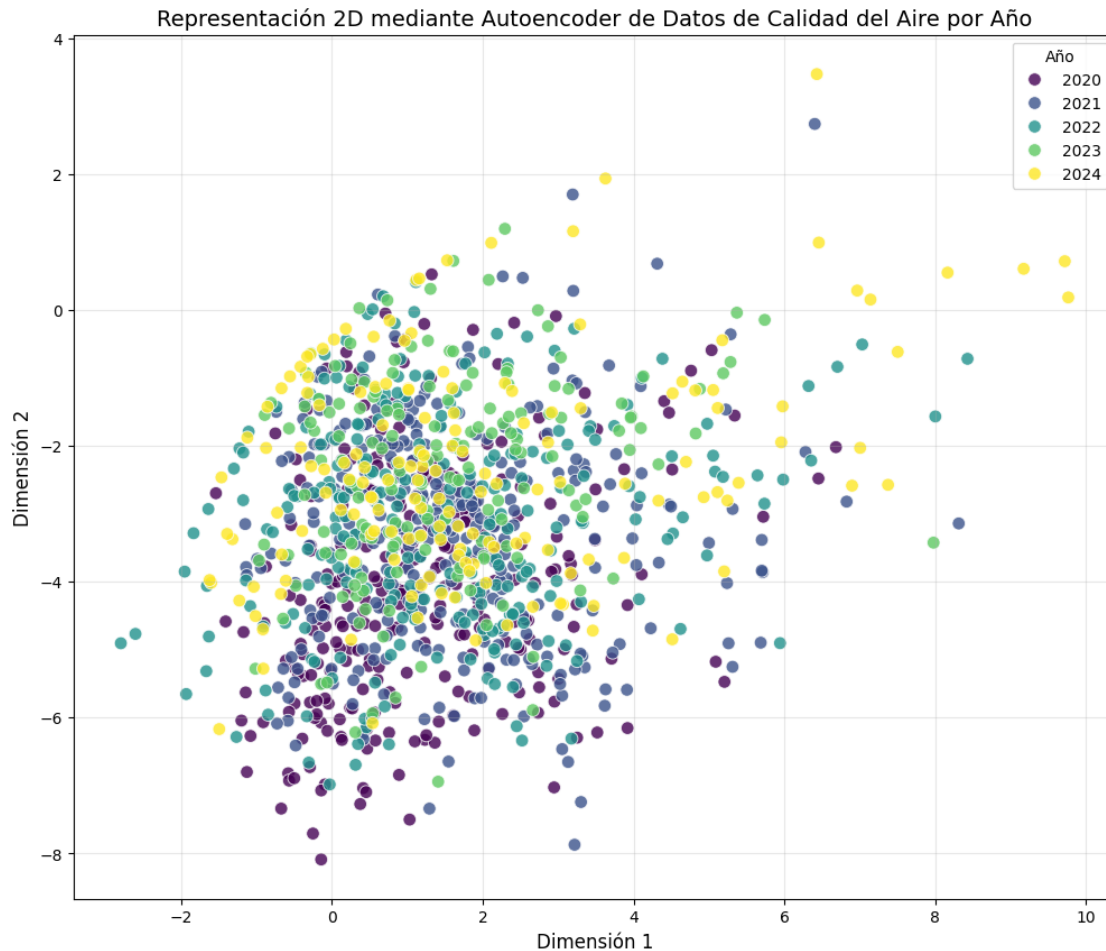
```
38/38 [=====] - 0s 676us/step
      x          y      date  year  month
```

0	2.896959	-2.848228	2020-01-18	2020	1
1	3.919591	-6.152839	2020-01-19	2020	1
2	5.205566	-5.471696	2020-01-20	2020	1
3	5.719744	-3.040446	2020-01-21	2020	1
4	3.517109	-1.753743	2020-01-22	2020	1

2.8 7. Visualizar la Representación 2D por Año

Vamos a crear un gráfico de dispersión de la representación 2D, coloreando los puntos por año para ver si hay patrones temporales.

```
[9]: # Crear un gráfico de dispersión con puntos coloreados por año
plt.figure(figsize=(12, 10))
sns.scatterplot(x='x', y='y', hue='year', palette='viridis', data=encoded_df,
               alpha=0.8, s=70)
plt.title('Representación 2D mediante Autoencoder de Datos de Calidad del Aire_
por Año', fontsize=14)
plt.xlabel('Dimensión 1', fontsize=12)
plt.ylabel('Dimensión 2', fontsize=12)
plt.legend(title='Año', fontsize=10)
plt.grid(True, alpha=0.3)
plt.show()
```

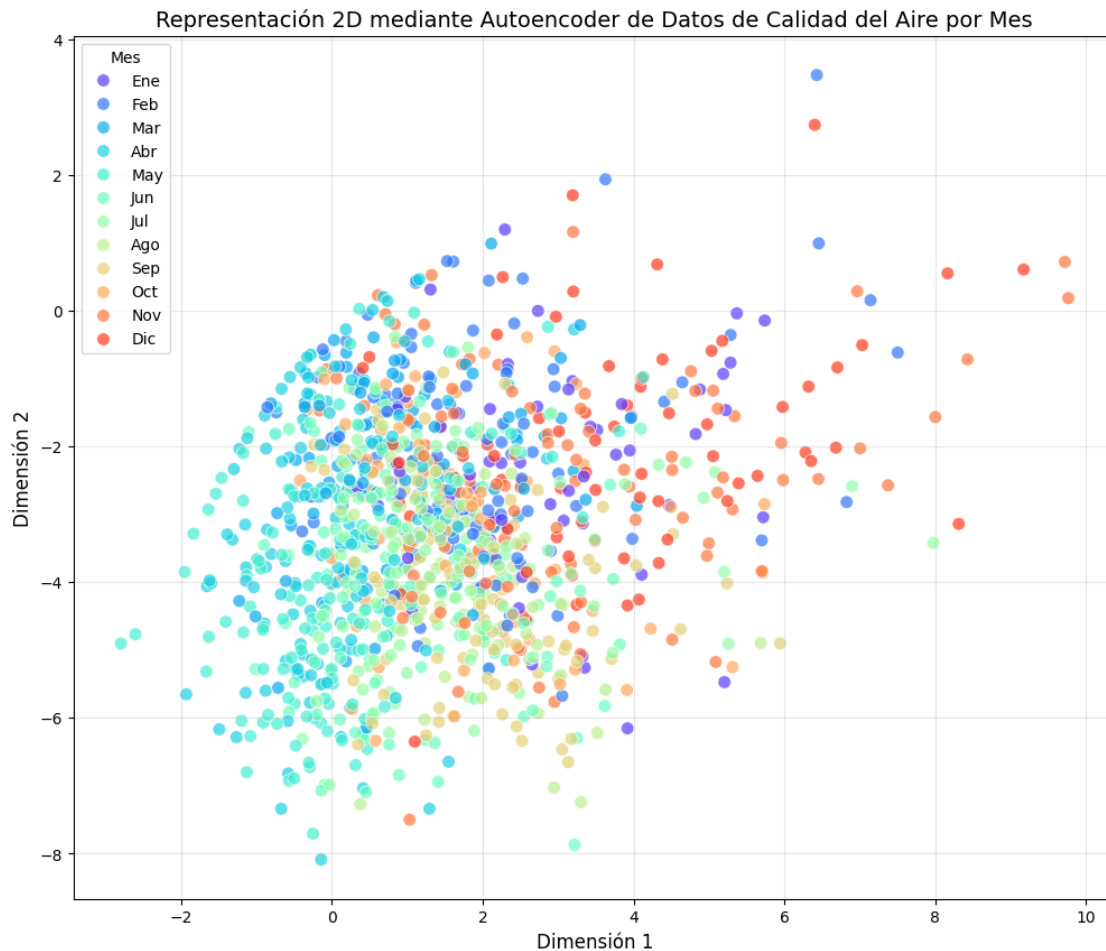
2.9 8. Visualizar la Representación 2D por Mes

Ahora vamos a colorear los puntos por mes para identificar patrones estacionales.

```
[10]: month_names = {1: 'Ene', 2: 'Feb', 3: 'Mar', 4: 'Abr', 5: 'May', 6: 'Jun',
                    7: 'Jul', 8: 'Ago', 9: 'Sep', 10: 'Oct', 11: 'Nov', 12: 'Dic'}
encoded_df['month_name'] = encoded_df['month'].map(month_names)

plt.figure(figsize=(12, 10))
sns.scatterplot(x='x', y='y', hue='month_name', palette='rainbow',
               data=encoded_df, alpha=0.7, s=70)
plt.title('Representación 2D mediante Autoencoder de Datos de Calidad del Aire_
           por Mes', fontsize=14)
plt.xlabel('Dimensión 1', fontsize=12)
plt.ylabel('Dimensión 2', fontsize=12)
plt.legend(title='Mes', fontsize=10)
plt.grid(True, alpha=0.3)
```

```
plt.show()
```



2.10 9. Analizar la Importancia de las Características mediante Reconstrucción Individual

Podemos analizar qué características son más importantes en la representación 2D observando el error de reconstrucción para cada característica.

```
[ ]: reconstructed_data = autoencoder.predict(scaled_data)

reconstruction_errors = np.mean(np.square(scaled_data - reconstructed_data),
    ↪axis=0)

# Crear un df con nombres de características y errores de reconstrucción
feature_importance = pd.DataFrame({
    'Feature': combined_daily_cleaned.columns,
    'Reconstruction Error': reconstruction_errors
```

```

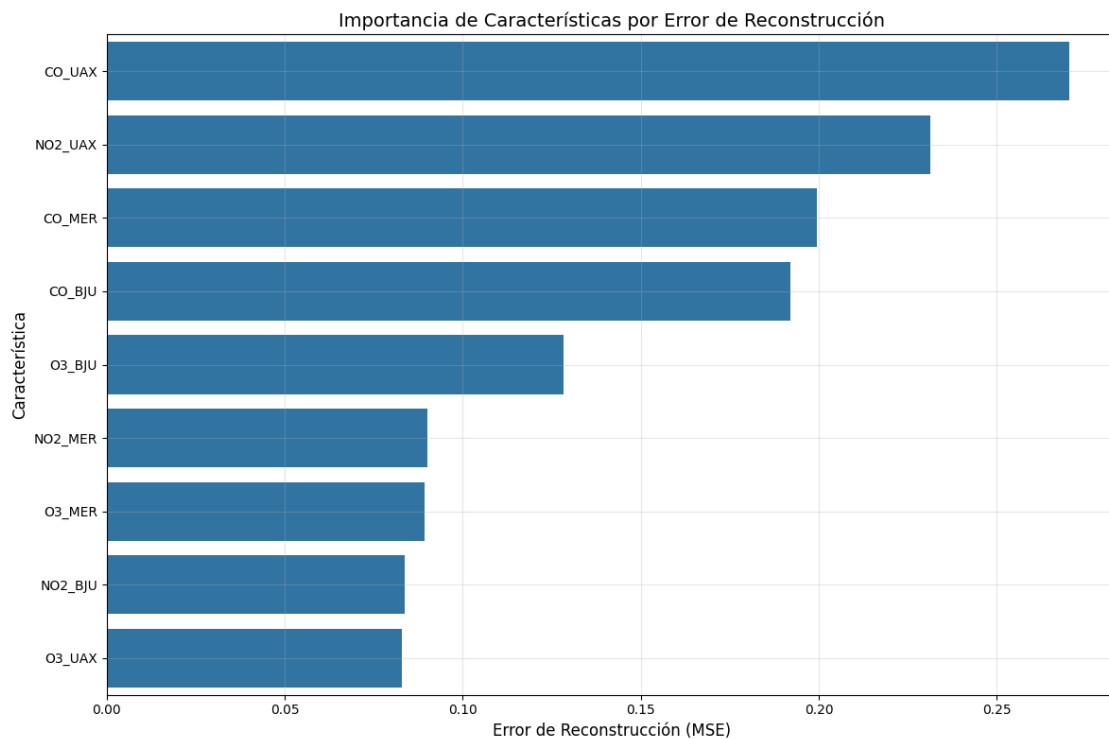
})

# Ordenar por error de reconstrucción (mayor error = menos bien representada en
↳ el espacio 2D)
feature_importance = feature_importance.sort_values('Reconstruction Error',
↳ ascending=False)

plt.figure(figsize=(12, 8))
sns.barplot(x='Reconstruction Error', y='Feature', data=feature_importance)
plt.title('Importancia de Características por Error de Reconstrucción',
↳ fontsize=14)
plt.xlabel('Error de Reconstrucción (MSE)', fontsize=12)
plt.ylabel('Característica', fontsize=12)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

38/38 [=====] - 0s 825us/step



2.11 10. Análisis de Correlación de Características Originales con Dimensiones 2D

Examinemos qué características originales se correlacionan con cada dimensión en nuestra representación 2D.

```
[ ]: correlation_df = pd.DataFrame(scaled_data, columns=combined_daily_cleaned.
    ↪columns)
correlation_df['dimension_1'] = encoded_data[:, 0]
correlation_df['dimension_2'] = encoded_data[:, 1]

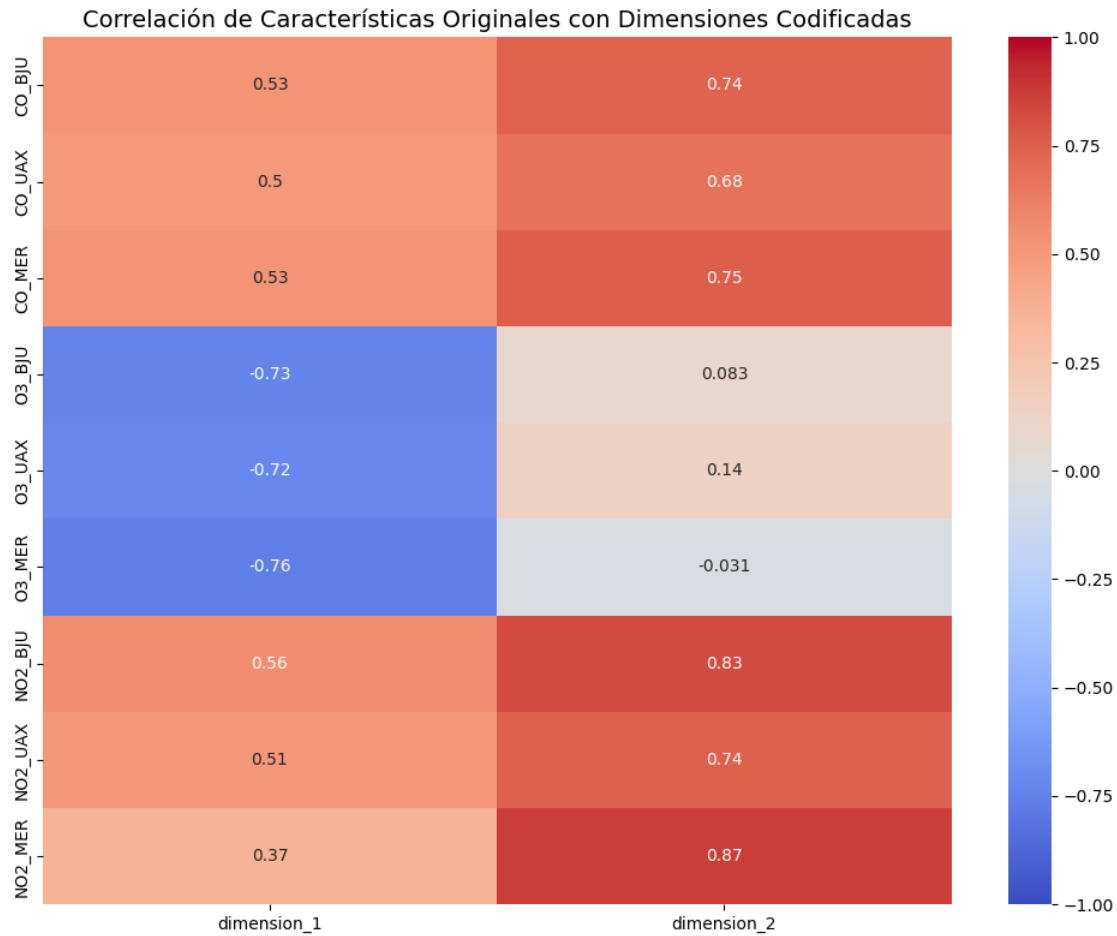
# Calcular correlación de características originales con dimensiones codificadas
correlations = correlation_df.corr()[['dimension_1', 'dimension_2']].iloc[:2]

plt.figure(figsize=(10, 8))
sns.heatmap(correlations, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Correlación de Características Originales con Dimensiones_
    ↪Codificadas', fontsize=14)
plt.tight_layout()
plt.show()

plt.figure(figsize=(12, 8))
plt.scatter(
    correlations['dimension_1'],
    correlations['dimension_2'],
    s=300,
    alpha=0.6
)

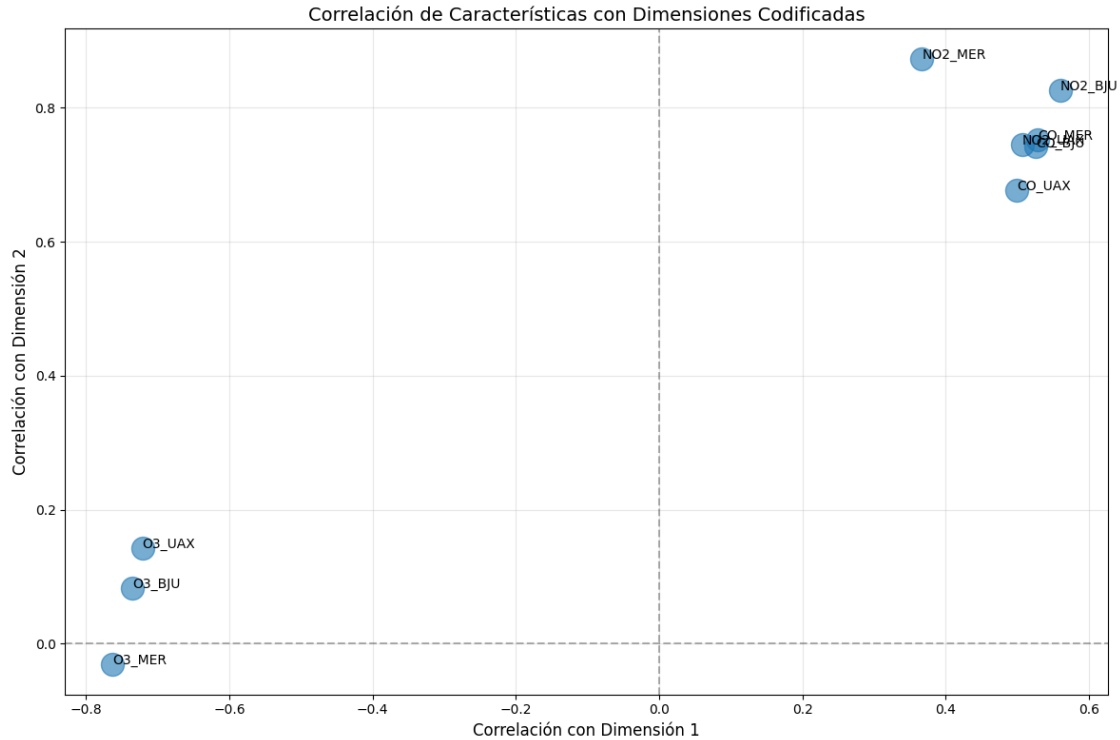
for i, txt in enumerate(correlations.index):
    plt.annotate(txt, (correlations['dimension_1'][i],
    ↪correlations['dimension_2'][i]), fontsize=10)

plt.axhline(y=0, color='k', linestyle='--', alpha=0.3)
plt.axvline(x=0, color='k', linestyle='--', alpha=0.3)
plt.grid(True, alpha=0.3)
plt.title('Correlación de Características con Dimensiones Codificadas',
    ↪fontsize=14)
plt.xlabel('Correlación con Dimensión 1', fontsize=12)
plt.ylabel('Correlación con Dimensión 2', fontsize=12)
plt.tight_layout()
plt.show()
```



C:\Users\thano\AppData\Local\Temp\ipykernel_8184\791724801.py:25: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`

```
plt.annotate(txt, (correlations['dimension_1'][i],
correlations['dimension_2'][i]), fontsize=10)
```



2.12 11. Visualización por estación

```
[ ]: correlation_matrix = correlation_df.corr()[['dimension_1', 'dimension_2']].
    ↪iloc[: -2]

stations = ['BJU', 'UAX', 'MER']
station_map = {}

for i, row in encoded_df.iterrows():
    max_correlation = -1
    dominant_station = None

    for station in stations:
        station_columns = [col for col in correlation_df.columns if station in
    ↪col and col not in ['dimension_1', 'dimension_2']]

        station_influence = 0
        for col in station_columns:
            if col in correlation_df.columns:
                station_influence += abs(correlation_df.loc[i, col]) * (
                    abs(correlation_matrix.loc[col, 'dimension_1']) *
    ↪abs(row['x']) +
```

```

        abs(correlation_matrix.loc[col, 'dimension_2']) *
↪abs(row['y'])
    )

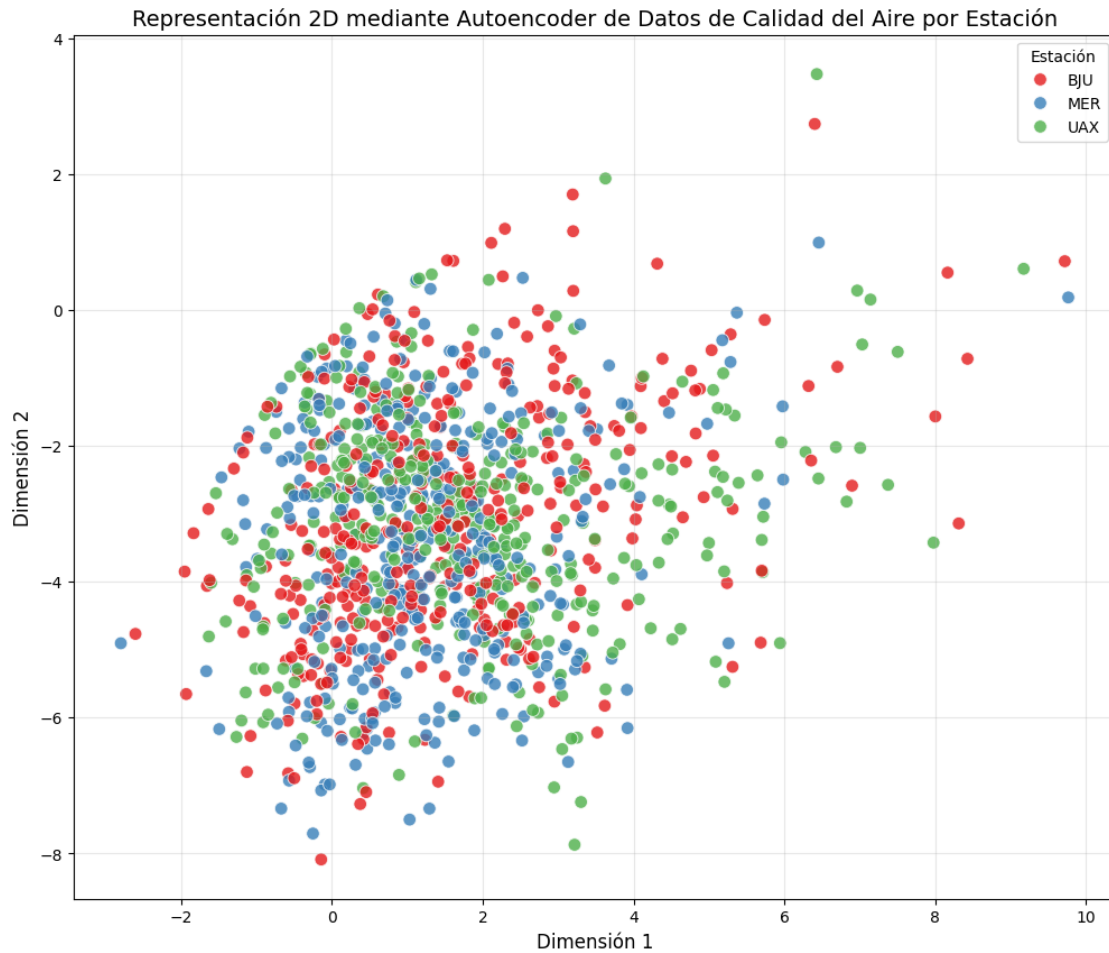
    if station_influence > max_correlation:
        max_correlation = station_influence
        dominant_station = station

    station_map[i] = dominant_station

encoded_df['station'] = encoded_df.index.map(lambda x: station_map.get(x,
↪'Unknown'))

plt.figure(figsize=(12, 10))
sns.scatterplot(x='x', y='y', hue='station', palette='Set1', data=encoded_df,
↪alpha=0.8, s=70)
plt.title('Representación 2D mediante Autoencoder de Datos de Calidad del Aire
↪por Estación', fontsize=14)
plt.xlabel('Dimensión 1', fontsize=12)
plt.ylabel('Dimensión 2', fontsize=12)
plt.legend(title='Estación', fontsize=10)
plt.grid(True, alpha=0.3)
plt.show()

```



2.13 12. Visualización por contaminante

```
[15]: # Visualización por tipo de contaminante
contaminants = ['CO', 'O3', 'NO2']
contaminant_map = {}

# Calcular cuál contaminante tiene mayor influencia en cada punto
for i, row in encoded_df.iterrows():
    max_correlation = -1
    dominant_contaminant = None

    for contaminant in contaminants:
        # Obtener columnas relacionadas con este contaminante
        contaminant_columns = [col for col in correlation_df.columns if col.
↪startswith(contaminant) and col not in ['dimension_1', 'dimension_2']]
```



```

    # Calcular la "influencia del contaminante" usando coordenadas y
    ↪ correlaciones
    contaminant_influence = 0
    for col in contaminant_columns:
        if col in correlation_df.columns:
            contaminant_influence += abs(correlation_df.loc[i, col]) * (
                abs(correlation_matrix.loc[col, 'dimension_1']) *
    ↪ abs(row['x']) +
                abs(correlation_matrix.loc[col, 'dimension_2']) *
    ↪ abs(row['y'])
            )

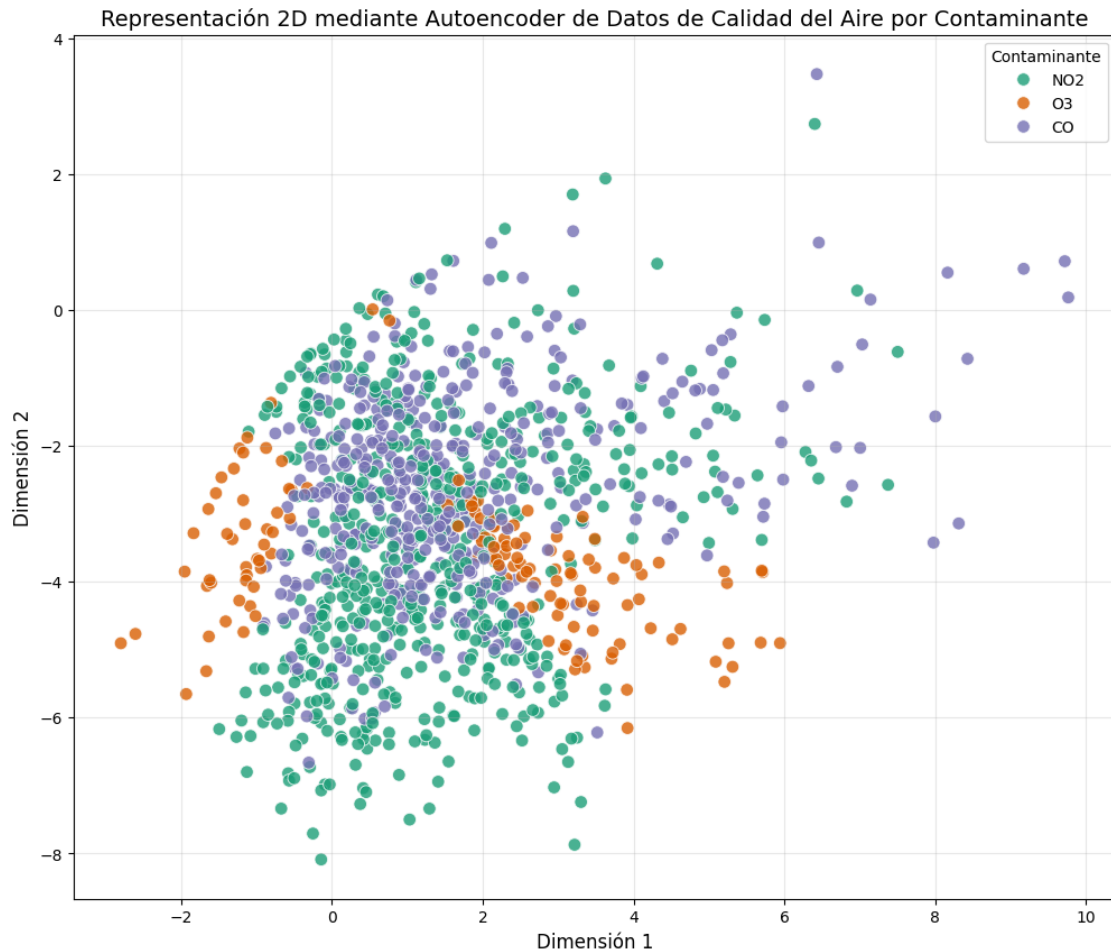
        if contaminant_influence > max_correlation:
            max_correlation = contaminant_influence
            dominant_contaminant = contaminant

    contaminant_map[i] = dominant_contaminant

# Añadir la información de contaminante al dataframe
encoded_df['contaminant'] = encoded_df.index.map(lambda x: contaminant_map.
    ↪ get(x, 'Unknown'))

# Crear un gráfico de dispersión con puntos coloreados por contaminante
plt.figure(figsize=(12, 10))
sns.scatterplot(x='x', y='y', hue='contaminant', palette='Dark2',
    ↪ data=encoded_df, alpha=0.8, s=70)
plt.title('Representación 2D mediante Autoencoder de Datos de Calidad del Aire
    ↪ por Contaminante', fontsize=14)
plt.xlabel('Dimensión 1', fontsize=12)
plt.ylabel('Dimensión 2', fontsize=12)
plt.legend(title='Contaminante', fontsize=10)
plt.grid(True, alpha=0.3)
plt.show()

```



2.14 13. Guardar el Modelo Codificador y los Datos Codificados

Finalmente, guardemos nuestro modelo codificador y los datos codificados para uso futuro.

```
[ ]: encoder_model.save('air_quality_encoder_model')
      print("Modelo codificador guardado.")

      encoded_df.to_csv('air_quality_encoded_data.csv', index=False)
      print("Datos codificados guardados en CSV.")
```

WARNING:tensorflow:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until you train or evaluate the model.

INFO:tensorflow:Assets written to: air_quality_encoder_model/assets

Modelo codificador guardado.

Datos codificados guardados en CSV.